DLOVS: Digital Land Ownership Verification System

Technical Report: Final Version of the Product | Monica Akoi Dau Ahol | BSc. Software Engineering, African Leadership University | Supervisor: Hubert Apana | 30 October 2026

Repository: https://github.com/Monica486-bot/dlovs **Web app:** https://dlovs-juba.netlify.app **API:** https://dlovs-api.onrender.com **Android APK:** https://github.com/Monica486-bot/dlovs/releases/download/v1.0.0/DLOVS-v1.0.apk **Video:** links document

DLOVS lets anyone in Juba check who legitimately owns a plot before money changes hands. A land officer registers each parcel and prints a cryptographically signed QR code for its marker; anyone can scan it or search the register without an account and see the registered owner, ownership history, disputes and verified documents. The system is three applications sharing one REST API: a Node.js/Express 5 API with PostgreSQL (Render), a React 19 web app for the public, citizens, land officers and administrators (Netlify), and a React Native/Expo Android app for citizens on the ground. All three are written in TypeScript and support English and Arabic.

1. Scope Alignment and Feature Validation

Of the 21 functional requirements in the approved proposal (Table 4), 20 are fully implemented and FR15 partially; the table groups them by feature. "Evidence" points to the repository; the README maps every requirement to its screens.

Planned feature	Intended purpose	Status	Evidence
Phone registration with SMS code (FR01, FR20)	Traceable accounts without email; recovery by SMS	Completed	authController.ts, phoneCodes.ts, sms.ts (Twilio); auth.test.ts
Guest verification, four pathways (FR02, FR06, FR07)	Remove the information gap before a sale	Completed	parcelController.ts verify/search; PublicVerify.tsx; mobile search.tsx; national ID search is staff-only
Parcel records (FR03)	Digital register created by officers only	Completed	RegisterParcel.tsx, EditParcel.tsx; role middleware
GPS identification with 10 m warning (FR04)	Confirm the buyer stands on the registered plot	Completed	mobile utils/location.ts, parcel/[id].tsx LocationCheck; LocationButton.tsx
Signed QR codes (FR05)	Markers that cannot be counterfeited	Completed	backend/utils/qr.ts (Ed25519); qrAndTransfer.test.ts
Append-only ownership history (FR08, NFR07)	Records that cannot be quietly rewritten	Completed	schema.sql triggers; auditTrail.test.ts
Owner portfolio (FR09)	Expose multiple-sale patterns	Completed	OwnerPortfolio.tsx, PublicOwner.tsx; mobile owner/[id].tsx
Document upload and verification (FR10, NFR10)	Digital backup of paper evidence	Completed	documentController.ts, documentStorage.ts (AES-256-GCM); 5 MB and type checks tested
Land transfer with new QR code (FR11)	Keep records current; stop double-selling	Completed	transferController.ts (owner consent, row locking, QR versioning); transferConsent.test.ts
Disputes and unregistered plots (FR12, FR13)	Warn buyers; route problems to officers	Completed	disputeController.ts, requestController.ts
Officer audit trail, separation of duties (FR14)	Officer accountability against corruption	Completed	audit_log + triggers; AuditLog.tsx; 20/20 test transactions logged
Notifications (FR15)	Tell owners about actions on their land	Partial	notify.ts, push.ts: in-app, SMS and Expo push; officers are not yet assigned jurisdictions
Offline cache, 50 records (FR16, NFR09)	Verification without connectivity	Completed	mobile utils/cache.tsx; offline QR check in utils/qr.ts
Search filters, low bandwidth (FR17, FR18)	Usable on 2G/3G	Completed	Text-only pages, about 200 KB per page, records under 1 KB
Bilingual interface (FR19)	English and Arabic speakers	Completed	web/src/i18n, mobile/src/i18n; right-to-left layout
Administrator dashboard (FR21)	Accounts, reports, fraud deactivation	Completed	Admin.tsx, UsageReports.tsx (CSV export)
Field evaluation (Objective 3: SUS, GPS triangulation, t-test)	Measure usability and impact in Gudele and Munuki	Not completed	Research activity outside the software build; the audit-capture metric (d) is verified by automated tests

Deviations from the proposal and their justification

Proposal	Built	Justification
SMS via Africa's Talking	Twilio	Africa's Talking's default Sender ID only reaches Airtel Kenya; +211 numbers need an approved dedicated Sender ID. One sendSms() function keeps the provider swappable.
Anyone searches by national ID	Staff only	A public ID search would reveal all of a person's land to anyone who knows their ID number.
QR signature checked on the phone with "crypto signing"	Ed25519 key pair	A shared secret in the APK could be extracted to forge codes; with Ed25519 the app holds only a public key, so the phone check (Figure 5 step 4) works offline and safely.
Officers alone process transfers	Owner consent required	Owner request, SMS code from the owner, or written consent approved by a second officer: one officer cannot move land alone.
Leaflet / react-native-maps	Text-only records	Map tiles are the heaviest load on 2G/3G; the GPS check compares positions directly.
Cloudinary at-rest encryption "to be confirmed"	AES-256-GCM in the API	Encryption no longer depends on a provider's plan; a leaked link yields only ciphertext.

Proposal	Built	Justification
JavaScript; Expo BarCodeScanner	TypeScript; expo-camera	Shared types catch API mistakes at build time; BarCodeScanner was removed from recent Expo SDKs.

Evidence: the deployed product

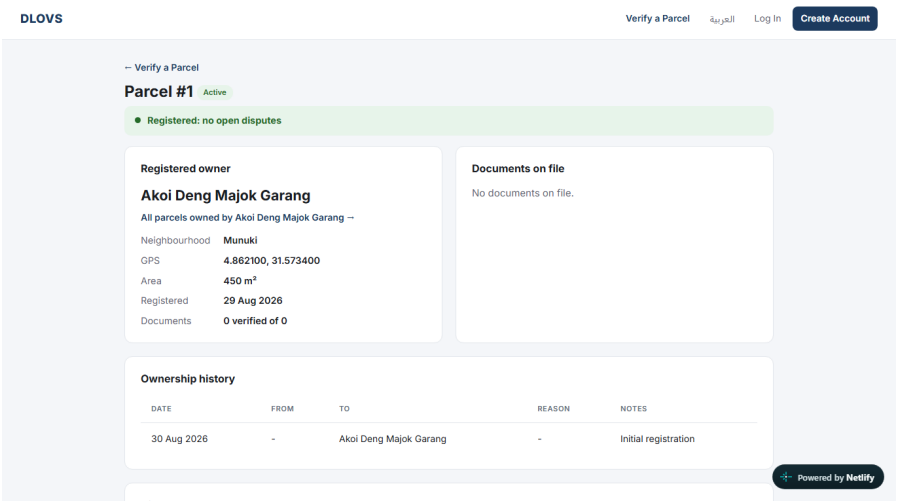


Figure 1. Live ownership record, Chrome 156 at 1366 x 768.

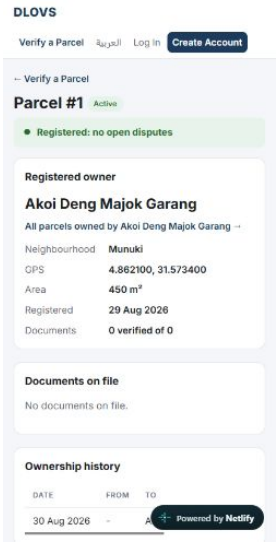


Figure 2. Same page on an emulated low-end phone (412 px).

2. Technical Functionality

Architecture. Both clients call the REST API over HTTPS; there is no direct database access. The API is layered: *routes* declare endpoints and attach middleware (authentication, role checks, rate limits, upload limits); *controllers* validate input and orchestrate; *utils* hold reusable services (QR signing, document encryption, SMS, push, notifications, password rules); PostgreSQL enforces integrity with constraints and triggers. JWT sessions carry the user's role; bcrypt hashes passwords.

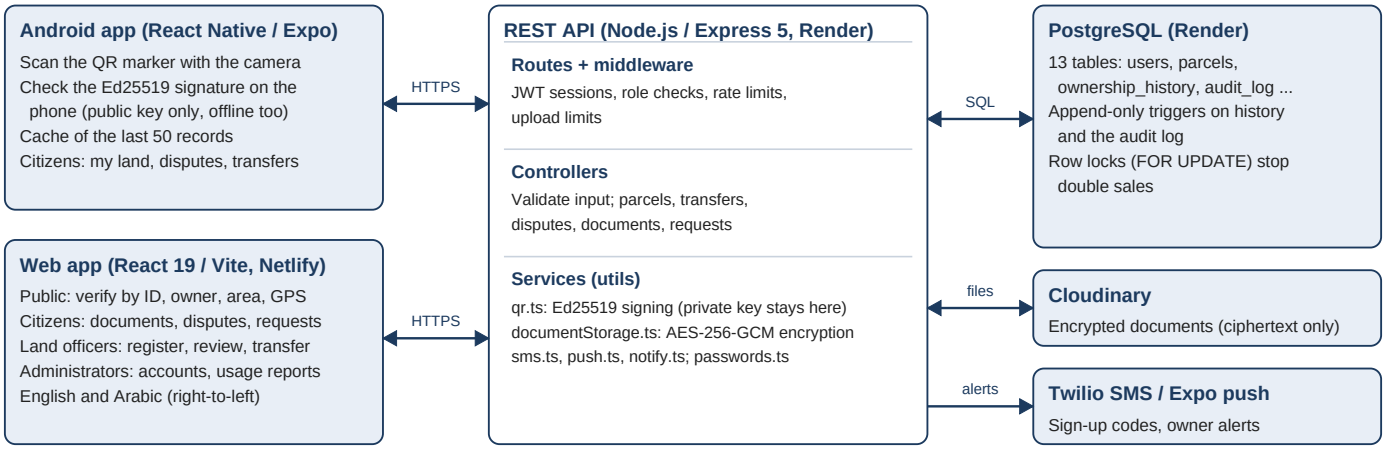


Figure 3. System architecture: both clients use one REST API; the private QR key, encryption and database access stay on the server, and the phone holds only the public key.

- **Verification flow.** The scanner reads */verify/<parcel>?v=<version>&s=<signature>*. The phone checks the Ed25519 signature with the stored public key (forgeries are refused even offline), then requests the record; the API re-checks the signature, returns owner, history, disputes and documents in one response (under 1 KB) and writes a VERIFY_PARCEL audit entry. Offline, the app shows its saved copy (last 50 records) and compares the marker's version.
- **Transfers.** A transaction locks the parcel row (SELECT ... FOR UPDATE), refuses disputed, deactivated or already-transferred parcels, appends history, increments the QR version and signs a new code. The previous owner's marker then reads as *replaced*, the signature of a resale.
- **Identity.** A national ID typed at sign-up is only linked to land after an officer has seen the ID card in person; a national ID belongs to one person, so land cannot be registered or transferred to someone else's ID under another name.
- **Data handling.** Documents are encrypted with AES-256-GCM before storage and checked by content type; SMS codes are stored hashed, expire after 10 minutes and lock after 5 attempts; secrets come only from environment variables.

3. Algorithms and Technical Problem-Solving

Problem	Approach	Outcome
GPS proximity search	Haversine great-circle distance, with an indexed latitude/longitude bounding-box pre-filter so the exact distance is computed only for nearby parcels; document counts only for the closest 50	500 m search on 50,000 parcels: 163 ms to 8 ms

Problem	Approach	Outcome
Unforgeable, versioned markers	Ed25519 signature over "DLOVS|parcel|version"; verification tries the claimed version and classifies valid / superseded / invalid	Offline forgery detection; resale detection
Double-sale prevention	Row-level locking inside a transaction; the second concurrent transfer sees the new owner and is refused	Tested with simultaneous transfers
Tamper-proof history	PostgreSQL triggers reject UPDATE, DELETE and TRUNCATE on ownership_history and audit_log	Enforced below the application
Secure one-time codes	SHA-256 hashed codes, constant-time comparison (timingSafeEqual), expiry, attempt limits and per-phone rate limiting	Brute force and timing attacks blocked
Password policy	NIST SP 800-63B: 8 to 72 bytes (bcrypt limit), block-list of common passwords, phone number refused, no forced character classes	One rule shared by every entry point
Offline cache	Most-recently-viewed list capped at 50 records in AsyncStorage	FR16 without unbounded storage

4. Testing Approaches and Results

Strategy	Why selected	How conducted	Results	Improvements made
Automated integration tests (Jest + Supertest)	Most risk is in API rules (roles, consent, signatures) that must stay correct as code changes	57 tests in 5 files against a real PostgreSQL test database rebuilt from schema.sql: valid and invalid inputs, edge cases (radius boundaries, expired codes, 5 MB limit, weak passwords, forged QR codes)	57/57 pass	Revealed a time-zone bug (API UTC+3, database UTC+2) that made every SMS code read as expired; sessions now run in UTC
Performance / load testing	NFR05 requires 50,000 parcels; NFR01 a record within 3 s on 3G and under 50 KB	Seeded 50,000 parcels and 40,000 owners; 50 timed requests per endpoint	Verify 11 ms, name search 45 ms, all responses under 12 KB	"Near me" scanned every parcel (163 ms); bounding box and index brought it to 8 ms
Compatibility testing (browsers, screens, emulated devices)	Users have low-end Android phones on slow networks; staff use desktop browsers	Lighthouse 12 in Chrome 156 and Edge 156 (Windows 11): emulated low-end phone (4x CPU slowdown, slow 4G) and desktop; screenshots at 412 px, 768 px, 1366 px and 1920 px	Accessibility 98-100, best practices 100; about 200 KB per page; layout fits at every size	Found a render-blocking font (about 0.7 s); now loaded without blocking. Remaining costs identified (see limitations)
Live deployment and device testing	The deployed system and APK must behave like the test build	Timed 20 live requests per endpoint; installed the APK on an OPPO A54 5G (Android 13, 4 GB RAM); scanned genuine, forged and non-DLOVS codes, online and offline; used the website in Chrome for Android	Live record in about 0.4 s and under 1 KB; all scan outcomes as expected	Removed an unneeded microphone permission from the APK; added offline QR verification after supervisor feedback
Manual functional testing	End-to-end flows across roles and languages	129 API checks and 28 browser steps as guest, citizen, two officers and administrator, in Arabic and at phone width	All passed	Fixed GPS badge shown when the phone reported no accuracy (FR04)

Link to scope. The automated tests cover FR01, FR02, FR03, FR05, FR06, FR08 to FR15 and NFR07 (sign-up, QR signatures, transfers, audit trail, documents, disputes, portfolio, plot reports, notifications). Requirements that depend on the device or the interface (FR04 GPS, FR16 offline cache, FR18 low bandwidth, FR19 Arabic, FR21 administration) were verified by manual, compatibility and on-device testing.

Evidence: backend/tests/, README "Testing", and docs/testing/compatibility.md (Lighthouse results, before/after table, screenshots, live timings and the real-device table).

5. Code Quality and OOP Practices

- **Modularity and separation of concerns.** Backend: 8 route modules, 8 controllers, 2 middleware and 12 utility services; web: 16 reusable components and 29 pages; mobile: shared UI components, utilities and screens per route. No file mixes HTTP handling, business rules and storage.
- **Encapsulation and abstraction.** Services expose small interfaces and hide their internals: qr.ts exports signParcelId, checkSignature and the public key while the private key never leaves the module; sendSms() hides the SMS provider; documentStorage.ts hides whether files go to Cloudinary or disk; notify.ts tracks pending notifications privately.
- **Inheritance and polymorphism.** Domain errors extend Error (CodeError, TransferError, ApiError, NetworkError) and carry HTTP status codes; callers handle them uniformly through instanceof, e.g. the mobile app falls back to the offline cache only on NetworkError.
- **Reusable components and types.** TypeScript interfaces describe every API response and are shared by the screens; UI building blocks (Banner, Card, ListItem, ParcelRecord sections) are reused across public, citizen and staff pages.
- **SOLID in practice.** Single responsibility per module; middleware composes cross-cutting checks (open for extension without editing controllers); controllers depend on service functions rather than providers (Twilio, Cloudinary) directly.
- **Maintainability.** Consistent naming (camelCase code, snake_case database), comments that explain *why* and cite the requirement (e.g. "FR04"), strict TypeScript type checking in all three apps, a linter (oxlint) on the web app, numbered database migrations, environment-based configuration, and a commit history that follows the features.

6. Limitations and Future Work

- The field evaluation with 40 participants (SUS score, GPS triangulation at five points, paired t-test) remains to be done in Juba.
- Free hosting: Render sleeps after inactivity (first request up to about a minute) and is in the USA (about 0.4 s per request from East Africa); Netlify's free plan injects a badge. Next: a Frankfurt region and a paid or badge-free host.
- The web app ships as one 134 KB script; splitting staff pages into a separate bundle would speed up public pages on low-end phones.
- Officers are not yet assigned jurisdictions (FR15); server messages are English only; national IDs are checked by eye, not against a national register.